

7 Chapter 7: Sorting

Sorting is used to arrange names and numbers in meaningful ways. Let A be a list of n elements $A_1, A_2, \dots, A_n$ in memory. Sorting of list A refers to the operation of rearranging the contents of A so that they are in increasing (or decreasing) order (numerically or lexicographically); $A_1 < A_2 < A_3 < \dots < A_n$.

There are many kinds of sorting techniques available in computer science. Some of them are as Bubble sort, Selection sort, Quick sort, Heap sort, Merge sort, Shell sort, Radix sort, Exchange sort, Internal sort, Insertion sort etc.

It is very difficult to select a sorting algorithm over another. And there is no sorting algorithm better than all others in all circumstances. Some sorting algorithm will perform well in some situations, so it is important to have a selection of sorting algorithms. Some factors that play an important role in selection processes are the time complexity of the algorithm (use of computer time), the size of the data structures (for Eg: an array) to be sorted (use of storage space), and the time it takes for a programmer to implement the algorithms (programming effort).

The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner. Sorting is also used to represent data in more readable formats.

Following are some of the examples of sorting in real-life scenarios

1. Telephone Directory – The telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.
2. Dictionary – The dictionary stores words in an alphabetical order so that searching of any word becomes easy.

7.1 In-place Sorting and Not-in-place Sorting

Sorting algorithms may require some extra space for comparison and temporary storage of few data elements. These algorithms do not require any extra space and sorting is said to happen in-place, or for example, within the array itself. This is called **in-place sorting**. Bubble sort is an example of in-place sorting.

However, in some sorting algorithms, the program requires space which is more than or equal to the elements being sorted. Sorting which uses equal or more space is called **not-in-place sorting**. Merge-sort is an example of not-in-place sorting.

7.2 Stable and Unstable Sorting

If a sorting algorithm, after sorting the contents, does not change the sequence of similar content in which they appear, it is called **stable sorting**.

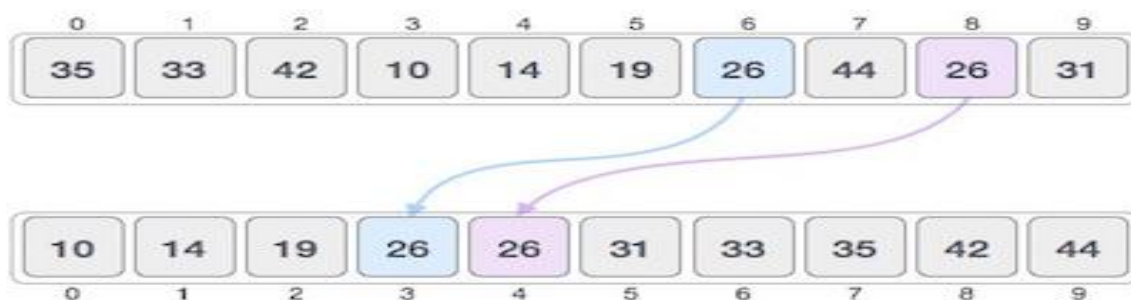


Figure 51: Stable Sort

If a sorting algorithm, after sorting the contents, changes the sequence of similar content in which they appear, it is called unstable sorting.

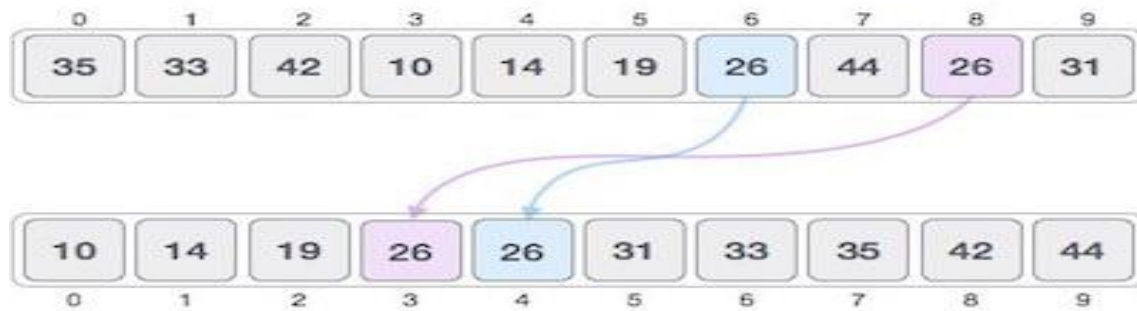


Figure 52: Unstable Sorting

Stability of an algorithm matters when we wish to maintain the sequence of original elements, like in a tuple for example.

7.3 Adaptive and Non-Adaptive Sorting Algorithm

A sorting algorithm is said to be **adaptive**, if it takes advantage of already 'sorted' elements in the list that is to be sorted. That is, while sorting if the source list has some element already sorted, adaptive algorithms will take this into account and will try not to re-order them.

A **non-adaptive** algorithm is one which does not take into account the elements which are already sorted. They try to force every single element to be re-ordered to confirm their sortedness.

7.3.1 Complexities of sorting algorithms

The complexity of sorting algorithm measures the running time of n items to be sorted. The operations in the sorting algorithm, where $A_1, A_2, \dots, A_n$ contains the items to be sorted and B is an auxiliary location, can be generalized as:

- (a) Comparisons- which tests whether $A_i < A_j$ or test whether $A_i < B$
- (b) Interchange- which switches the contents of A_i and A_j or of A_i and B
- (c) Assignments- which set $B = A$ and then set $A_j = B$ or $A_j = A_i$

Normally, the complexity functions measure only the number of comparisons, since the number of other operations is at most a constant factor of the number of comparisons.

7.4 BUBBLE SORT

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. Then next element is compared with its adjacent element and the same process is repeated for all the elements in the array until we get a sorted array. This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where n is the number of items.

Example:

We take an unsorted array for our example.



Pass	Input Array	Comparison	Status	Swap/No Swap	New array
1	14,33,27,35,10	14 > 33	False	No Swap	14,33,27,35,10
	14,33,27,35,10	33 > 27	True	Swap	14,27,33,35,10
	14,27,33,35,10	33 > 35	False	No Swap	14,27,33,35,10
	14,27,33,35,10	35 > 10	True	Swap	14,27,33,10,35
2	14,27,33,10,35	14 > 27	False	No Swap	14,27,33,10,35
	14,27,33,10,35	27 > 33	False	No Swap	14,27,33,10,35

	14,27, 33 ,10,35	33 > 10	True	Swap	14,27,10,33,35
3	14 ,27,10,33,35	14 > 27	False	No Swap	14,27,10,33,35
	14, 27 ,10,33,35	27 > 10	True	Swap	14,10,27,33,35
4	14 ,10,27,33,35	14 > 10	True	Swap	10,14,27,33,35

Table 4: Bubble sort

Bubble sort requires **n-1** passes to sort an entire array. In pass 1, largest number will be moved to highest index. And in pass 2, two largest elements will be sorted and so on.

Bubble sort starts with very first two elements, comparing them to check which one is greater.

Pass 1:

Step 1: Compare 14 and 33, Value 33 is greater than 14, so it is already in sorted locations.

Step 2: Compare 33 with 27. In this case, Value 27 is smaller than 33 and these two values must be swapped. And the new array should look like this: **14,27,33,35,10**

Step 3: Compare 33 and 35. We find that both are in already sorted positions.

Step 4: Compare 35 and 10. In this case, Value 10 is smaller than 35 and these two values must be swapped, and the new array should look like this: **14,27,33,10,35**

Pass 2:

Step 1: Compare 14 and 27, Value 27 is greater than 14, so it is already in sorted locations.

Step 2: Compare 27 and 33, Value 33 is greater than 27, so it is already in sorted locations.

Step 3: Compare 33 and 10, Value 10 is smaller than 33, and these two values must be swapped, and the new array should look like this: **14,27,10,33,35**

Pass 3:

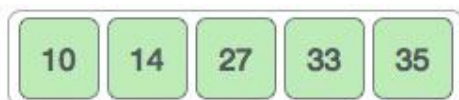
Step 1: Compare 14 and 27, Value 27 is greater than 14, so it is already in sorted locations.

Step 2: Compare 27 and 10, Value 10 is smaller than 27, and these two values must be swapped, and the new array should look like this: **14,10,27,33,35**

Pass 4:

Step 1: Compare 14 and 10, Value 10 is smaller than 24, and these two values must be swapped, and the new array should look like this: **10,14,27,33,35**

The final sorted list is:

**7.4.1 Algorithm**

Let A be a linear array of n numbers. tmp is a temporary variable for swapping (or interchange) the position of the numbers.

1. Input n numbers of an array A
2. Initialize i = 0 and repeat through step 4 if (i < n-1)
3. Initialize j = 0 and repeat through step 4 if (j < n - 1 - i)
4. If (A[j] > A[j + 1])
 - i. tmp = A[j]
 - ii. A[j] = A[j + 1]
 - iii. A[j + 1] = tmp
5. Display the sorted numbers of array A
6. Exit.

Source Code:

```
/* Bubble sort*/
```

```

void bubble_sort(int a[],int n)
{
    int i,j,tmp;
    for (i = 0; i <n-1; i++)
    {
        for (j = 0; j <n-1-i; j++)
        {
            if (arr[j] > arr[j+1])
            {
                tmp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = tmp;
            } //End of if
        } //End of inner for loop
    } //End of outer for loop
} //End of function

```

7.4.2 Time Complexity

Inner loop executes for (n-1) times when i=0, (n-2) times when i=1 and so on:

Time complexity = (n-1) + (n-2) + (n-3) + +2 +1
 = $O(n^2)$

There is no best-case linear time complexity for this algorithm.

7.5 SELECTION SORT

- Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end, which are divided by an imaginary wall.
- Initially, the sorted part is empty and the unsorted part is the entire list.
- The smallest element is selected from the unsorted sub list and swap it with the element at the beginning of the unsorted data.
- After each selection and swapping, the imaginary wall between the two sub lists move one element ahead, increasing the number of sorted elements and decreasing the number of unsorted ones.
- Each time we move one element from the unsorted sub list to the sorted sub list, we say that we have completed a sort pass.
- A list of n elements requires **n-1** passes to completely rearrange the data.
- This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where n is the number of items.

Example:

Consider the following depicted array as an example.



Initially set the min index to 0.

Pass	Iteration	Input Array	Min Index	Comparison	Status	Update Status	New Array
1	1	14,33,27,10,35,19,42,44	0	14 > 33	False	No update	
	2	14,33,27,10,35,19,42,44	0	14 > 27	False	No update	
	3	14,33,27,10,35,19,42,44	0	14 > 10	True	Set Min= 3	
	4	14,33,27,10,35,19,42,44	3	10 > 35	False	No update	
	5	14,33,27,10,35,19,42,44	3	10 > 19	False	No update	
	6	14,33,27,10,35,19,42,44	3	10 > 42	False	No Update	

	7	14,33,27,10,35,19,42,44	3	10 > 44	False	Swap A[0] and A[3]	10,33,27,14,35,19,42,44
2	1	10,33,27,14,35,19,42,44	1	33 > 27	True	Set Min=2	
	2	10,33,27,14,35,19,42,44	2	27 > 14	True	Set Min=3	
	3	10,33,27,14,35,19,42,44	3	14 > 35	False	No Update	
	4	10,33,27,14,35,19,42,44		14 > 19	False	No Update	
	5	10,33,27,14,35,19,42,44		14 > 42	False	No Update	
	6	10,33,27,14,35,19,42,44		14 > 44	False	Swap A[1] and A[3]	10,14,27,33,35,19,42,44
3	1	10,14,27,33,35,19,42,44	2	27 > 33	False	No Update	
	2	10,14,27,33,35,19,42,44	2	27 > 35	False	No Update	
	3	10,14,27,33,35,19,42,44	2	27 > 19	True	Set Min=5	
	4	10,14,27,33,35,19,42,44	5	19 > 42	False	No Update	
	5	10,14,27,33,35,19,42,44	5	19 > 44	False	Swap A[2] and A[5]	10,14,19,33,35,27,42,44
4	1	10,14,19,33,35,27,42,44	3	33 > 35	False	No Update	
	2	10,14,19,33,35,27,42,44	3	33 > 27	True	Set Min=5	
	3	10,14,19,33,35,27,42,44	5	27 > 42	False	No Update	
	4	10,14,19,33,35,27,42,44	5	27 > 44	False	Swap A[3] and A[5]	10,14,19,27,35,33,42,44
5	1	10,14,19,27,35,33,42,44	4	35 > 33	True	Set Min=5	
	2	10,14,19,27,35,33,42,44	5	33 > 42	False	No Update	
	3	10,14,19,27,35,33,42,44	5	33 > 44	False	Swap A[4] and A[5]	10,14,19,27,33,35,42,44
6	1	10,14,19,27,33,35,42,44	5	35 > 42	False	No Update	
	2	10,14,19,27,33,35,42,44	5	35 > 44	False	No Swap	10,14,19,27,33,35,42,44
7	1	10,14,19,27,33,35,42,44	6	42 > 44	False	No Swap	10,14,19,27,33,35,42,44

Table 5: Selection Sort

7.5.1 Algorithm

Let A be a linear array of n numbers. tmp be a temporary variable for swapping (or interchanging). min is the variable to store the location of smallest number

1. Input n numbers of an array A
2. Initialize i = 0 and repeat through step5 if (i < n - 1)
 - i. min = i
3. Initialize j = i + 1 and repeat through step 4 if (j < n)
4. if (min > a[j])
 - i. min = j
5.
 - i. tmp = a[min]
 - ii. a[min] = a[i]
 - iii. a[i] = tmp
6. Display "the sorted numbers of array A"
7. Exit

Source Code:

```
void selectionsort(int arr[],int n)
{
    int i,j,temp;
    int min;
    for(i=0;i<n-1;i++)
    {
        min = i;
        for(j=i+1;j<n;j++)
        {
            if(arr[min] > arr[j])
                min = j;
        }
    }
}
```

```

    }
    temp= arr[min];
    arr[min]=arr[i];
    arr[i]=temp;
  }
}

```

7.5.2 Time Complexity:

Inner loop executes for (n-1) times when i=0, (n-2) times when i=1 and so on:

Time complexity = (n-1) + (n-2) + (n-3) + +2 +1
 = $O(n^2)$

There is no best-case linear time complexity for this algorithm, but number of swap operations are reduced greatly.

7.6 INSERTION SORT

Insertion sort algorithm sorts a set of values by inserting values into an existing sorted file. Compare the second element with first, if the first element is greater than second, place it before the first one. Otherwise place is just after the first one. Compare the third value with second. If the third value is greater than the second value then place it just after the second. Otherwise place the second value to the third place. And compare third value with the first value. If the third value is greater than the first value place the third value to second place, otherwise place the first value to second place. And place the third value to first place and so on.

The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-list (in the same array). This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$, where **n** is the number of items.

Let A be a linear array of n numbers A [1], A [2], A [3], A[n]. The algorithm scans the array A from A [1] to A [n], by inserting each element A [k], into the proper position of the previously sorted sub list. A [1], A [2], A [3], A [k - 1]

Step 1: As the single element A [1] by itself is sorted array.

Step 2: A [2] is inserted either before or after A [1] by comparing it so that A[1], A[2] is sorted array.

Step 3: A [3] is inserted into the proper place in A [1], A [2], that is A [3] will be compared with A [1] and A [2] and placed before A [1], between A [1] and A [2], or after A [2] so that A [1], A

[2], A [3] is a sorted array.

Step 4: A [4] is inserted in to a proper place in A [1], A [2], A [3] by comparing it; so that A [1], A [2], A [3], A [4] is a sorted array.

Step 5: Repeat the process by inserting the element in the proper place in array

Step n : A [n] is inserted into its proper place in an array A [1], A [2], A [3], A [n - 1] so that A [1], A [2], A [3], ,A [n] is a sorted array.

Example:

Consider the following depicted array as an example.



Iteration	Input Array	Comparison	Status	Update Status	New Array
1	14,33,27,10,35,19,42,44	14 > 33	False	Initial List Sorted	14,33,27,10,35,19,42,44
2	14,33,27,10,35,19,42,44	33 > 27	True	Swap 33 and 27	14,27,33,10,35,19,42,44
2	14,27,33,10,35,19,42,44	14 > 27	False	Sublist sorted	14,27,33,10,35,19,42,44
3	14,27,33,10,35,19,42,44	33 > 10	True	Swap 33 and 10	14,27,10,33,35,19,42,44
3	14,27,10,33,35,19,42,44	27 > 10	True	Swap 27 and 10	14,10,27,33,35,19,42,44
3	14,10,27,33,35,19,42,44	14 > 10	True	Swap 14 and 10	10,14,27,33,35,19,42,44

4	14,10,27,33,35,19,42,44	33 > 35	False	Sub list Sorted	10,14,27,33,35,19,42,44
5	10,14,27,33,35,19,42,44	35 > 19	True	Swap 35 and 19	10,14,27,33,19,35,42,44
5	10,14,27,33,19,35,42,44	33 > 19	True	Swap 33 and 19	10,14,27,19,33,35,42,44
5	10,14,27,19,33,35,42,44	27 > 19	True	Swap 27 and 19	10,14,19,27,33,35,42,44
5	10,14,19,27,33,35,42,44	19 > 14	False	Sub List sorted	10,14,19,27,33,35,42,44
6	10,14,19,27,33,35,42,44	35 > 42	False	Sub List Sorted	10,14,19,27,33,35,42,44
7	14,10,27,33,35,19,42,44	42 > 44	False	Sub List Sorted	10,14,19,27,33,35,42,44

Table 6: Insertion Sort

Note : **[Bold in New Array column shows the sorted sub list.]**

1. Insertion sort compares the first two elements.
2. It finds that both 14 and 33 are already in ascending order. For now, 14 is in sorted sub-list.
3. Insertion sort moves ahead and compares 33 with 27. And finds that 33 is not in the correct position. It swaps 33 with 27. It also checks with all the elements of sorted sub-list. Here we see that the sorted sub-list has only one element 14, and 27 is greater than 14. Hence, the sorted sub-list remains sorted after swapping.
4. By now we have 14 and 27 in the sorted sub-list. Next, it compares 33 with 10. These values are not in a sorted order. So we swap them.
5. However, swapping makes 27 and 10 unsorted. Hence, we swap them too.
6. Again we find 14 and 10 in an unsorted order. We swap them again.
7. This process goes on until all the unsorted values are covered in a sorted sub-list.

7.6.1 Algorithm

Let A be a linear array of n numbers A [1], A [2], A [3], ,A [n],tmp be a temporary variable to interchange the two values. Pos is the control variable to hold the position of each pass.

1. Input an array A of n numbers
2. Initialize i = 1 and repeat through steps 4 by incrementing i by one.
 - i. If (i <= n - 1)
 - ii. tmp = A [i],
 - iii. Pos = i - 1
3. Repeat the step 3 if (tmp < A[Pos] and (Pos >= 0))
 - iv. A [Pos+1] = A [Pos]
 - v. Pos = Pos-1
4. A [Pos +1] = tmp
5. Exit

Source Code:

```
/*Insertion sort*/
void insertion_sort(int arr[], int n)
{
    int i,j,k;
    for(i=1;i<n;i++)
    {
        k=arr[i]; /*k is to be inserted at proper place*/
        for(j=i-1;j>=0 && k<arr[j];j--)
            arr[j+1]=arr[j];
        arr[j+1]=k;
    }
}
```

7.7 SHELL SORT

Shell sort is a highly efficient sorting algorithm and is based on insertion sort algorithm. Shell Sort is introduced to improve the efficiency of simple insertion sort. Shell Sort is also called diminishing

increment sort. In this method, sub-array, contain kth element of the original array, are sorted. This algorithm avoids large shifts as in case of insertion sort, if the smaller value is to the far right and has to be moved to the far left.

This algorithm uses insertion sort on a widely spread elements, first to sort them and then sorts the less widely spaced elements. This spacing is termed as interval.

This algorithm is quite efficient for medium-sized data sets as its average and worst case complexity are of $O(n)$, where n is the number of items.

Let A be a linear array of n numbers $A[1], A[2], A[3], \dots, A[n]$.

Step 1: The array is divided into k sub-arrays consisting of every k th element. Say $k=5$, then five sub-array, each containing one fifth of the elements of the original array.

Sub array 1 $\rightarrow A[0] A[5] A[10]$

Sub array 2 $\rightarrow A[1] A[6] A[11]$

Sub array 3 $\rightarrow A[2] A[7] A[12]$

Sub array 4 $\rightarrow A[3] A[8] A[13]$

Sub array 5 $\rightarrow A[4] A[9] A[14]$

Note : The i^{th} element of the j^{th} sub array is located as $A[(i-1) * k + j - 1]$

Step 2: After the first k sub array are sorted (usually by insertion sort) , a new smaller value of k is chosen and the array is again partitioned into a new set of sub arrays.

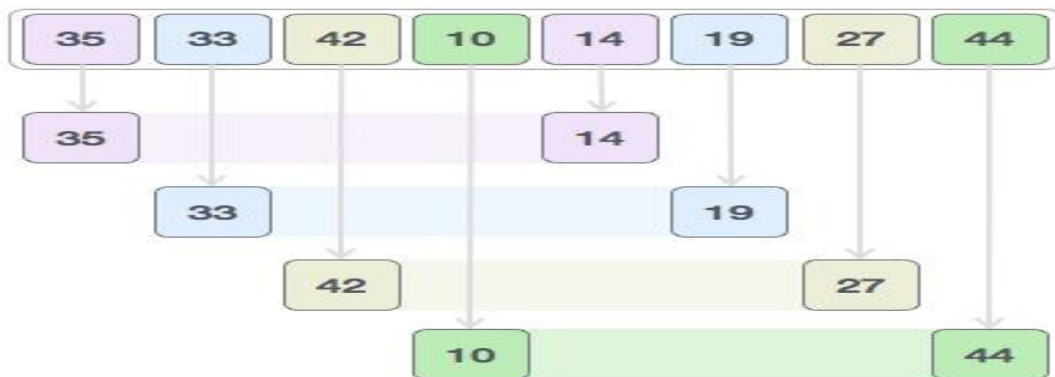
Step 3: And the process is repeated with an even smaller value of k , so that $A[1], A[2], A[3], \dots, A[n]$ is sorted.

Example:

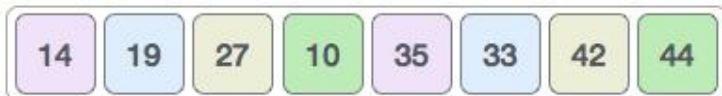
Consider the following array with 8 elements 35,33,42,10,14,19,27,44 and the span $K = 4, 2, 1$ is chosen.

Pass 1:

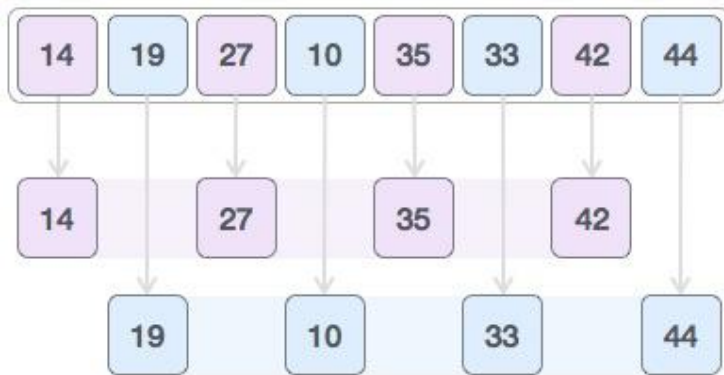
Make a sub-list of all values located at the interval of 4 positions. Here these values are {35, 14}, {33, 19}, {42, 27} and {10, 44}



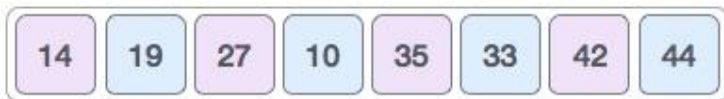
We compare values in each sub-list and swap them (if necessary) in the original array. After this step, the new array should look like this:



Then, we take interval of 2 and this gap generates two sub-lists - {14, 27, 35, 42}, {19, 10, 33, 44}



We compare and swap the values, if required, in the original array. After this step, the array should look like this:



Finally, we sort the rest of the array using interval of value 1. Shell sort uses insertion sort to sort the array.

Following is the step-by-step depiction:

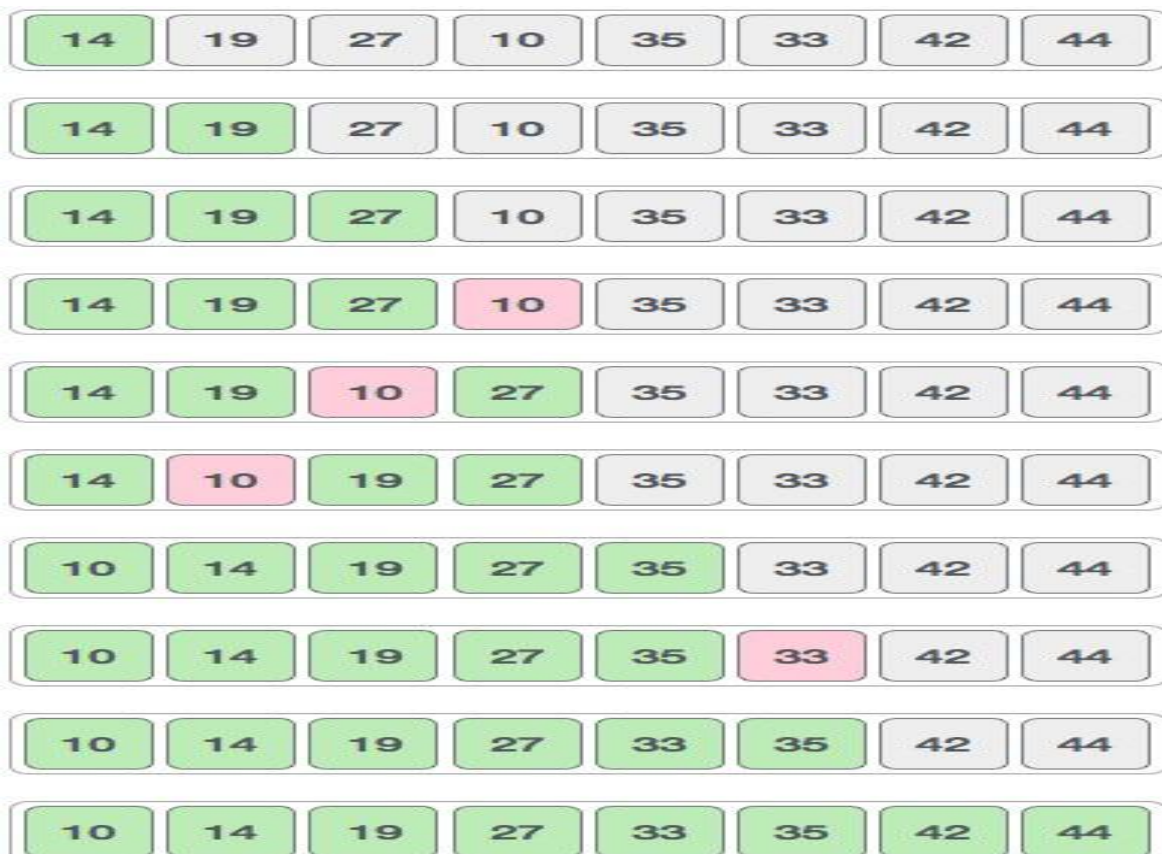


Figure 53: Shell Sort

We see that it required only four swaps to sort the rest of the array.

7.7.1 ALGORITHM

Let A be a linear array of n elements, A [1], A [2], A [3], A[n] and Incr be an array of sequence of span.

1. Input n numbers of an array A
2. Input incr and repeat through step 5 until (incr >=1)
3. Initialize j = incr and repeat through step 5 until (j < n)
(a) k = A [j]
4. Initialize i = j-incr and repeat through step 5 if (i >= 0) and (k < A [i])
(a) A [i + incr] = A [i]
5. A [i + incr] = k
6. Exit

Source Code:

```
void shell_sort(int arr[],int n)
{
    int incr, i,j,k,
    scanf("%d",&incr);
    while(incr>=1)
    {
        for (j=incr;j<n;j++)
        {
            k=arr[j];
            for(i=j-incr; i >= 0 && k < arr[i]; i = i-incr)
                arr[i+incr]=arr[i];
            arr[i+incr]=k;
        }
        incr=incr-2; /*Decrease the increment*/
    } /*End of while*/
}
```

7.8 Quick Sort

It is one of the widely used sorting techniques and it is also called the partition exchange sort. Quick sort is an efficient algorithm and it passes a very good time complexity in average case. It is an algorithm of the **divide-and conquer** type. The quick sort algorithm works by partitioning the array to be sorted. And each partition is internally sorted recursively. This key value can be the first element of an array. That is, if A is an array then key = A[0], and rest of the elements are grouped into two portions such that,

- a) One partition contains elements smaller than key value.
- b) Another partition contains elements larger than key value.

Two pointers, Up and Low, are initialized to the upper and lower bounds of the sub array. During execution, at any point each element in a position above up is greater than or equal to key value and each element in a position below low pointer is less than or equal to the key. Up pointer will move in a decrement and low in an increment fashion.

Let A be an array A[1], A[2], A[3],....A[n] of n numbers to be sorted. Then

Step1: Choose the first elements of the array(or sub-array) as the key i.e. key = A[1].

Step2: Place the low pointer in second position of the array and up pointer in the last position of the array i.e. low = 2 and up = n.

Step3: Repeatedly increase the pointer low by one position until A[low] > key.

Step4: Repeatedly decrease the pointer up by one position until A[up] <=key.

Step5: If up > low, interchange A[low] with A[up].

Step6: Repeat the step 3,4,5 until the step 5 fails.(i.e. up <=low)

Note: The given array is partition into two sub arrays. The sub array A[1] A[2].....A[k-1] is less than A[k] i.e. key and the second sub array A[k+1] A[k+2].....A[n] which is greater than a key value A[k].

Lets take an example of the following data and i and j as the low and up pointer.

40 20 10 80 60 50 7 30 100

```
enum bool pivot_placed=FALSE;
//setting the pointers
left=low;
right=up;
piv=low; /*Take the first element of sublist as piv */
if (low>=up)
    return;
printf ("\nSublist : ");
display(arr,low,up);
/*Loop till pivot is placed at proper place in the sublist*/
while(pivot_placed==FALSE)
{
    /*Compare from right to left */
    while( arr[piv]<=arr[right] && piv!=right )
        right=right-1;
    if ( piv==right )
        pivot_placed=TRUE;
    if ( arr[piv] > arr[right] )
    {
        temp=arr[piv];
        arr[piv]=arr[right];
        arr[right]=temp;
        piv=right;
    }
    /*Compare from left to right */
    while( arr[piv]>=arr[left] && left!=piv )
        left=left+1;
    if (piv==left)
        pivot_placed=TRUE;
    if ( arr[piv] < arr[left] )
    {
        temp=arr[piv];
        arr[piv]=arr[left];
        arr[left]=temp;
        piv=left;
    }
} /*End of while */
printf ("-> Pivot Placed is %d -> ",arr[piv]);
display(arr,low,up);
printf ("\n");
quick(arr,low,piv-1);
quick(arr,piv+1,up);
} /*End of quick() */
int main()
{
    int array[MAX],n,i;
    printf ("\nEnter the number of elements : ");
    scanf ("%d",&n);
    for (i=0;i<n;i++)
    {
        printf ("\nEnter element %d : ",i+1);
        scanf ("%d",&array[i]);
    }
    printf ("\nUnsorted list is :\n");
    display(array,0,n-1);
    printf ("\n");
    quick (array,0,n-1);
```

```
printf ("\nSorted list is :\n");
display(array,0,n-1);
return 0;
}/*End of main() */
```

7.8.2 TIME COMPLEXITY

The time complexity of quick sort can be calculated for any of the following case. It is usually measured by the number $f(n)$ of comparisons required to sort n elements.

7.8.2.1 WORST CASE

The worst case occurs when the list is already sorted. In this case the given array is partitioned into two sub arrays. One of them is an empty array and another one is an array. After partition, when the first element is checked with other element, it will take n comparison to recognize that it remain in the position so as $(n - 1)$ comparisons for the second position.

$$\begin{aligned}f(n) &= n + (n - 1) + \dots + 2 + 1 \\&= (n(n + 1))/2 \\&= O(n^2)\end{aligned}$$

7.8.2.2 AVERAGE CASE

In this case each reduction step of the algorithm produces two sub arrays. Accordingly:

- Reducing the array by placing one element and produces two sub arrays.
- Reducing the two sub-arrays by placing two elements and produces four subarrays.
- Reducing the four sub-arrays by placing four elements and produces eight subarrays.

And so on. Reduction step in the k^{th} level finds the location at 2^{k-1} element; however there will be approximately $\log_2 n$ levels at reduction steps. Furthermore, each level uses at most n comparisons, so

$$f(n) = O(n \log n)$$

7.8.2.3 BEST CASE

The base case analysis occurs when the array is always partitioned in half, That key

$$= A[(\text{low} + \text{up})/2]$$

$$f(n) = Cn + f(n/2) + f(n/2)$$

$$= Cn + 2f(n/2)$$

$$= O(n) \text{ where } C \text{ is a constant.}$$

7.9 MERGE SORT

Merging is the process of combining two or more sorted array into a third sorted array. Divide the array into approximately $n/2$ sub-arrays of size two and set the element in each sub array. Since at any time the two arrays being merged are both sub-arrays of A , lower and upper bounds are required to indicate the sub-arrays of a being merged. $l1$ and $u1$ represents the lower and upper bands of the first sub-array and $l2$ and $u2$ represents the lower and upper bands of the second sub-array respectively. Let A be an array of n number of elements to be sorted $A[1], A[2] \dots A[n]$.

Step 1: Divide the array A into approximately $n/2$ sorted sub-array of size 2. i.e., the elements in the $(A[1], A[2]), (A[3], A[4]), (A[k], A[k + 1]), (A[n - 1], A[n])$ sub-arrays are in sorted order.

Step 2: Merge each pair of pairs to obtain the following list of sorted sub-array of size 4; the elements in the sub-array are also in the sorted order. $(A[1], A[2], A[3], A[4]), \dots (A[k - 1], A[k], A[k + 1], A[k + 2]), \dots (A[n - 3], A[n - 2], A[n - 1], A[n])$.

Step 3: Repeat the step 2 recursively until there is only one sorted array of size.

Example: $a[] = \{4, 7, 2, 6, 1, 4, 7, 3, 5, 2, 6\}$

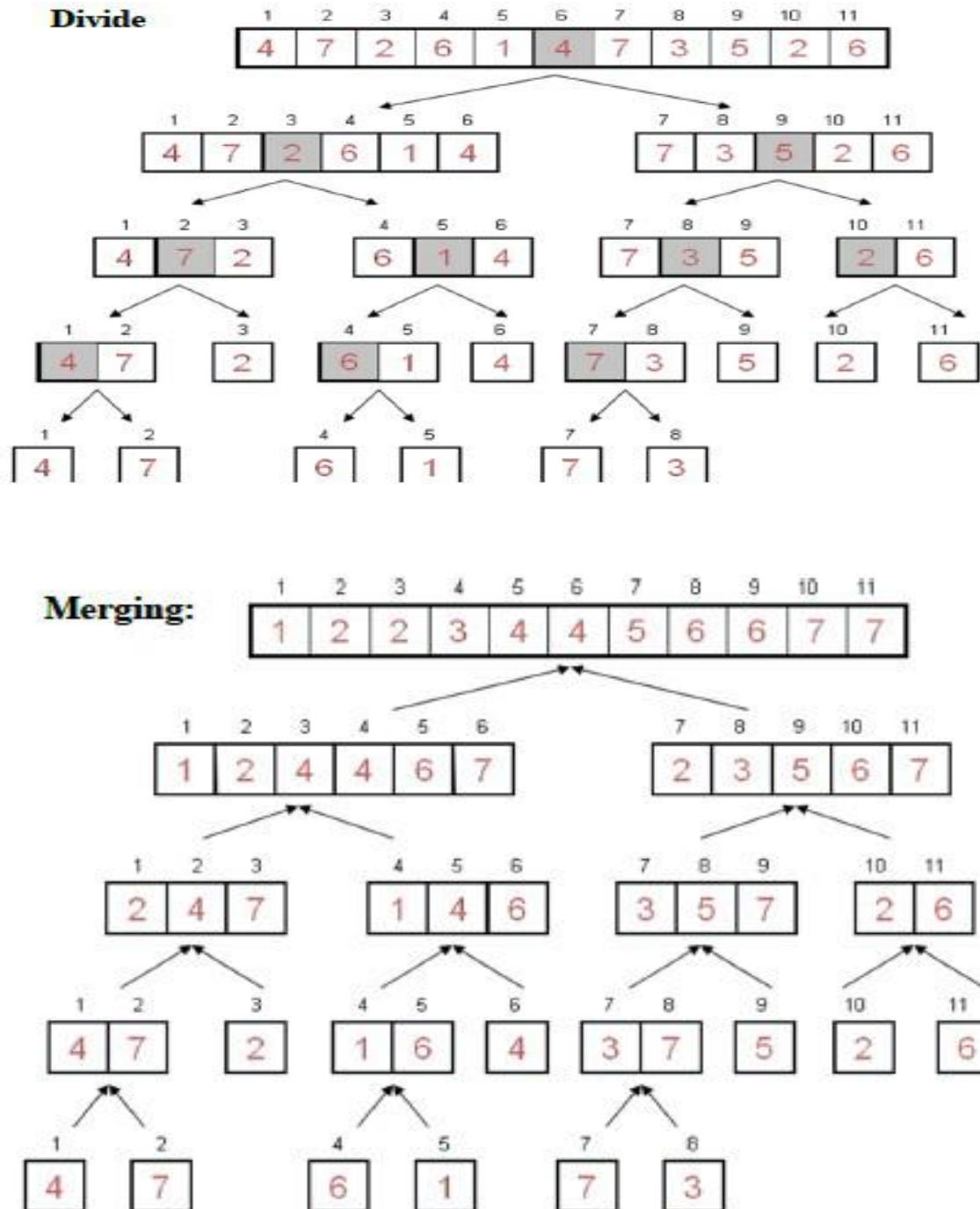


Figure 54: Merge Sort

7.9.1 Algorithm of Merge sort

Input: Array $A[1..N]$, indices p, q, r ($p \leq q < r$). $A[p..r]$ is the array to be divided. $A[p]$ is the beginning element and $A[r]$ is the ending element

Output: Array $A[p..r]$ in ascending order

MERGE-SORT(A, p, q, r)

1. if $p < r$

2. then $q \leftarrow (r + p)/2$
3. MERGE-SORT(A, p, q)
4. MERGE-SORT(A, q+1, r)
5. MERGE(A, p, q, r)

MERGE(A, p, q, r)

6. $n1 \leftarrow q - p + 1$
7. $n2 \leftarrow r - q$
8. create arrays L[1...N1+1] and R[1...N2+1]
9. for $i \leftarrow 1$ to $N1$
10. do $L[i] \leftarrow A[p+i-1]$
11. for $j \leftarrow 1$ to $n2$
12. do $R[j] \leftarrow A[q+j]$
13. $L[N1+1] \leftarrow \infty$
14. $R[N2+1] \leftarrow \infty$
15. $i \leftarrow 1$
16. $j \leftarrow 1$
17. for $k \leftarrow p$ to r
18. do if $L[i] \leq R[j]$
19. then $A[k] \leftarrow L[i]$
20. $i \leftarrow i+1$
21. else $A[k] \leftarrow R[j]$
22. $j \leftarrow j+1$

Source Code:

```
#include<stdio.h>
#define MAX 20
int array[MAX];
//Function to merge the sub files or arrays
void merge(int low, int mid, int high )
{
    int temp[MAX];
    int i = low;
    int j = mid +1 ;
    int k = low ;
    while( (i <= mid) && (j <=high) )
    {
        if (array[i] <= array[ j])
            temp[k++] = array[i++] ;
        else
            temp[k++] = array[ j++] ;
    }/*End of while*/
    while( i <= mid )
        temp[k++] = array[i++];
    while( j <= high )
        temp[k++] = array[j++];
    for (i = low; i <= high ; i++)
        array[i] = temp[i];
}/*End of merge()*/
//Function which call itself to sort an array
void merge_sort(int low, int high )
{
    int mid;
    if ( low != high )
    {
```

```
        mid = (low+high)/2;
        merge_sort( low , mid );
        merge_sort( mid+1, high );
        merge(low, mid, high );
    }
}/*End of merge_sort*/
int main()
{
    int i,n;
    printf ("\nEnter the number of elements : ");
    scanf ("%d",&n);
    for (i=0;i<n;i++)
    {
        printf ("\nEnter element %d : ",i+1);
        scanf ("%d",&array[i]);
    }
    printf ("\nUnsorted list is :\n");
    for ( i = 0 ; i<n ; i++)
    printf ("%d ", array[i]);
    merge_sort( 0, n-1);
    printf("\nSorted list is :\n");
    for ( i = 0 ; i<n ; i++)
    printf ("%d ", array[i]);
    printf("\n");
    return 0;
}/*End of main()*/
```

7.9.2 TIME COMPLEXITY

Let $f(n)$ denote the number of comparisons needed to sort an n element array A using the merge sort algorithm. The algorithm requires almost $\log n$ passes, each involving n or fewer comparisons. In average and worst case the merge sort requires $O(n \log n)$ comparisons. The main drawback of merge sort is that it requires $O(n)$ additional space for the auxiliary array.

7.10 Radix Sort

Radix sort or bucket sort is a method that can be used to sort a list of numbers by its base. It is also called as **clever sort**. If the key is an integer number, each position is a digit and has ten possibilities: 0 . . 9. If the key is a string of letters and case is not important, then each position has 26 possibilities: "a" . . "z". The number of possibilities is called the **radix**.

To sort an array of decimal numbers, where the radix or base is 10, we need 10 buckets and can be numbered as 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Number of passes required to have a sorted array depends upon the number of digits in the largest element. To illustrate the radix sort, consider the following array with 7 elements:

42, 133, 7, 23, 74, 670, 49

In this array the biggest element is 670 and the number of digit is 3. So 3 passes are required to sort the array. Read the element(s) and compare the first position (2 is in first position of 42) digit with the digit of the bucket and place it.

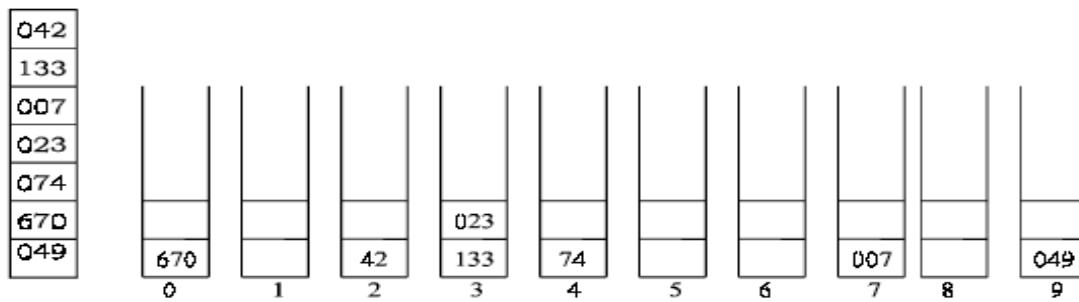


Figure 55: Radix sort Pass 1

Now read the elements from left to right and bottom to top of the buckets and place it in array for the next pass. Read the array element and compare the second position (4 is in second position of the element 042) digit with the number of the bucket and place it.

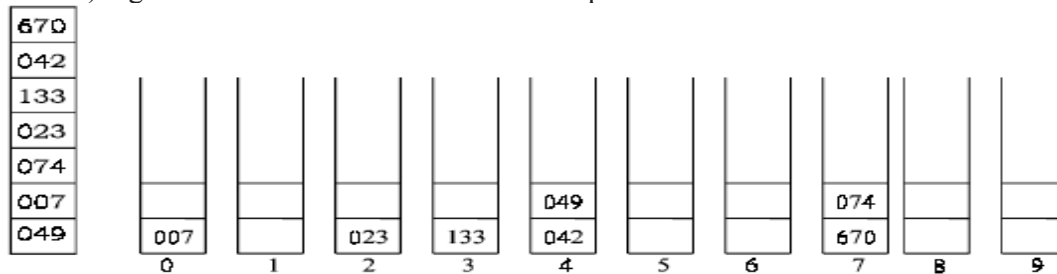


Figure 56: Radix Sort Pass 2

Again read the element from left to right and from bottom to top to get an array for the third pass. (0 is in the third position of 042) Compare the third position digit in each element with the bucket digit and place it wherever it matches.

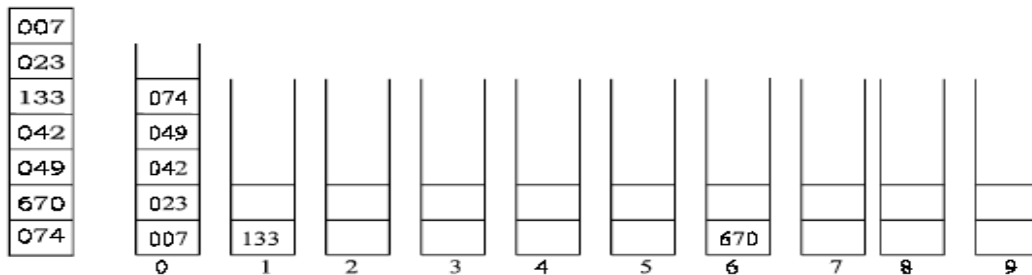


Figure 57: Radix Sort Pass 3

Read the element from the bucket from left to right and from bottom to top for the sorted array. i.e., 7 23, 42, 49, 74, 133, 670.

7.10.1 Algorithm (Radix Sort)

Let A be a linear array of n elements A[1], A[2], A[3], A[n]. Digit is the total number of digits in the largest element in array A.

1. Input n number of elements in an array A.
2. Find the total number of Digits in the largest element in the array.
3. Initialize i = 1 and repeat the steps 4 and 5 until (i <= Digit).
4. Initialize the buckets j = 0 and repeat the steps (a) until (j < n)
 - (a) Compare *i*th position of each element of the array with bucket number and place it in the corresponding bucket.
5. Read the element(s) of the bucket from 0th bucket to 9th bucket and from first position to higher one to generate new array A.
6. Display the sorted array A.
7. Exit.

Source Code: Using Linked List

```
#include<stdio.h>
#include "stdlib.h"
struct node
{
    int info ;
    struct node *link;
}*start=NULL;
//Display the array elements
void display()
{
    struct node *p=start;
    while( p !=NULL)
    {
        printf ("%d ", p->info);
        p= p->link;
    }
    printf ("\n");
}/*End of display()*/
/* This function finds number of digits in the largest element of
the list */
int large_dig(struct node *p)
{
    int large = 0,ndig = 0 ;
    while (p != NULL)
    {
        if (p ->info > large)
            large = p->info;
        p = p->link ;
    }
    printf ("\nLargest Element is %d ,",large);
    while (large != 0)
    {
        ndig++;
        large = large/10 ;
    }
    printf ("\nNumber of digits in it are %d\n",ndig);
    return(ndig);
} /*End of large_dig()*/
/*This function returns kth digit of a number*/
int digit(int number, int k)
{
    int digit, i ;
    for (i = 1 ; i <=k ; i++)
    {
        digit = number % 10 ;
        number = number /10 ;
    }
    return(digit);
}/*End of digit()*/
//Function to implement the radix sort algorithm
void radix_sort()
{
    int i,k,dig,maxdig,mindig,least_sig,most_sig;
    struct node *p, *rear[10], *front[10];
    least_sig=1;
    most_sig=large_dig(start);
```

```
for (k = least_sig; k <= most_sig ; k++)
{
    printf ("\nPASS %d : Examining %dth digit from right",k,k);
    for(i = 0 ; i <= 9 ; i++)
    {
        rear[i] = NULL;
        front[i] = NULL ;
    }
    maxdig=0;
    mindig=9;
    p = start ;
    while( p != NULL)
    {
        /*Find kth digit in the number*/
        dig = digit(p->info, k);
        if (dig>maxdig)
            maxdig=dig;
        if (dig<mindig)
            mindig=dig;
        /*Add the number to queue of dig*/
        if (front[dig] == NULL)
            front[dig] = p ;
        else
            rear[dig]->link = p ;
        rear[dig]= p ;
        p=p->link; /*Go to next number in the list*/
    } /*End while */
    /* maxdig and mindig are the maximum amd minimum
       digits of the kth digits of all the numbers*/
    printf ("\nmindig=%d maxdig=%d\n",mindig,maxdig);
    /*Join all the queues to form the new linked list*/
    start=front[mindig];
    for (i=mindig;i<maxdig;i++)
    {
        if (rear[i+1]!=NULL)
            rear[i]->link=front[i+1];
        else
            rear[i+1]=rear[i];
    }
    rear[maxdig]->link=NULL;
    printf ("\nNew list :");
    display();
} /* End for */
} /*End of radix_sort*/

int main()
{
    struct node *tmp,*q;
    int i,n,item;
    printf ("\nEnter the number of elements in the list : ");
    scanf ("%d", &n);
    for (i=0;i<n;i++)
    {
        printf ("\nEnter element %d :",i+1);
        scanf ("%d",&item);
        /* Inserting elements in the linked list */
        tmp=(struct node*)malloc(sizeof(struct node));
        tmp->info=item;
    }
}
```

```

tmp->link=NULL;
if (start==NULL) /* Inserting first element */
    start=tmp;
else
{
    q=start;
    while (q->link!=NULL)
        q=q->link;
    q->link=tmp;
}
}/*End of for*/
printf ("\nUnsorted list is :\n");
display();
radix_sort();
printf ("\nSorted list is :\n");
display ();
}/*End of main() */

```

7.11 Heap

A heap is defined as an almost complete binary tree of n nodes such that the value of each node is less than or equal to the value of the father. It can be sequentially represented as

$$A[j] \leq A[(j-1)/2]$$

$$\text{for } 0 \leq [(j-1)/2] < j \leq n-1$$

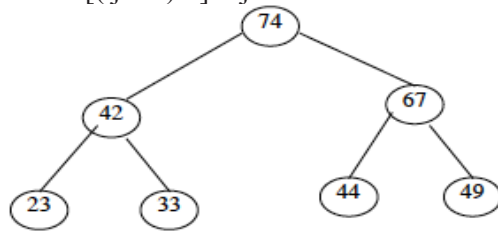


Figure 58: Heap



Figure 59: Array representation of Heap

The root of the binary tree (i.e., the first array element) holds the largest key in the heap. This type of heap is usually called descending heap or **max heap**. We can also define an ascending heap as an almost complete binary tree in which the value of each node is greater than or equal to the value of its father. This root node has the smallest element of the heap. This type of heap is also called **min heap**.

Priority Queue: A priority queue is an ADT with the following operations:

- Insert a new element into the queue
- Find the element with the largest key
- Delete the element with the largest key

Other common operations:

- Heaps provide an efficient implementation of priority queues:
- get the maximum $\rightarrow$ take the root
- delete the maximum $\rightarrow$ move the last element to the root and heapify.
- insert a new element $\rightarrow$ put it at the end and raise it until it's in place

7.11.1 Heap Sort

A heap can be used to sort a set of elements. Let H be a heap with n elements and it can be sequentially

represented by an array A. Inset an element data into the heap H as follows:

1. First place data at the end of H so that H is still a complete tree, but not necessarily a heap.
2. Then the data be raised to its appropriate place in H so that H is finally a heap.

7.11.1.1 Inserting an element in to a heap

Consider the heap H in Fig1. Say we want to add a data = 55 to H.

Step 1: First we adjoin 55 as the next element in the complete tree as shown in Fig3. Now we have to find the appropriate place for 55 in the heap by rearranging it.

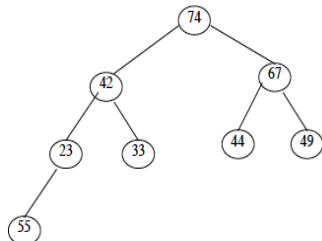


Figure 60: Heap sort (1)

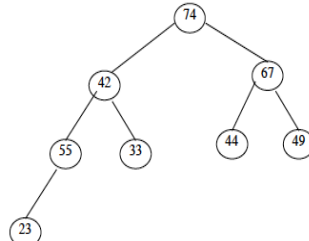


Figure 61: Heap sort (2)

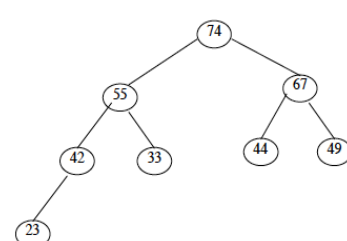


Figure 62: Heap sort (3)

Step 2: Compare 55 with its parent 23. Since 55 is greater than 23, interchange 23 and 55. Now the heap will look like as in Fig4.

Step 3: Compare 55 with its parent 42. Since 55 is greater than 42, interchange 55 and 42. Now the heap will look like as in Fig5.

Step 4: Compare 55 with its new parent 74. Since 55 is less than 74, it is the appropriate place of node 55 in the heap H.

7.11.1.2 Deleting the Root of a Heap

Let H be a heap with n elements. The root R of H can be deleted as follows:

Step1: Assign the root R to some variable data.

Step2: Replace the deleted node R by the last node (or recently added node) L of H so that H is still a complete tree, but not necessarily a heap.

Step3: Now rearrange H in such a way by placing L (new root) at the appropriate place, so that H is finally a heap.

Consider the heap H in Fig5 where R = 74 is the root and L = 23 is the last node (or recently added node) of the tree. Suppose we want to delete the root node R = 74. Apply the above rules to delete the root. Delete the root node R and assign it to data (i.e., data = 74) as shown in Fig6.

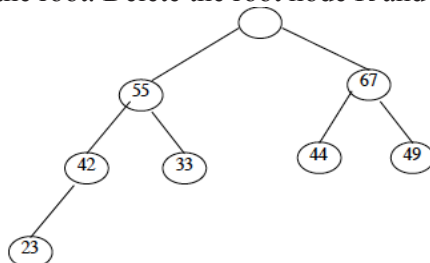


Figure 63: Heap Sort (4)

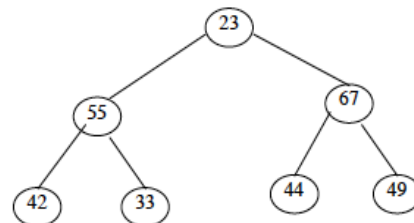


Figure 64: Heap Sort (5)

Replace the deleted root node R by the last node L as shown in the Fig7. Compare 23 with its new two children 55 and 67. Since 23 is less than the largest child 67, interchange 23 and 67. The new tree looks like as in Fig8. Again compare 23 with its new two children, 44 and 49. Since 23 is less than the largest child 49, interchange 23 and 49 as shown in Fig9.

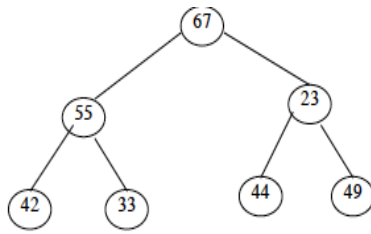


Figure 65: Heap Sort (6)

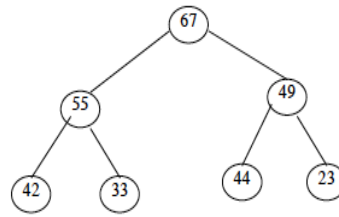


Figure 66: Heap Sort (7)

The Fig70 is the required heap H without its original root R.

7.11.1.3 Algorithm

Let H be a heap with n elements stored in the array HA. Data is the item of the node to be removed. Last gives the information about last node of H. The LOC, left, right gives the location of Last and its left and right children as the Last rearranges in the appropriate place in the tree.

1. Input n elements in the heap H
2. Data = HA[0]; last = HA[n-1] and n = n - 1
3. LOC = 0, left = 2*LOC+1 and right = 2*LOC+2
4. Repeat the steps 5, 6 and 7 while (right <= n)
5. If (last >= HA[left]) and (last >= HA[right])
 - (a) HA[LOC] = last
 - (b) Exit
6. a) If (HA[right] <= HA[left])
 - (i) HA[LOC] = HA[left]
 - (ii) LOC = left
- (b) Else
 - (i) HA[LOC] = HA[right]
 - (ii) LOC = right
7. left = 2 * LOC; right = left + 1
8. If (left = n) and (last < HA[left])
 - (a) LOC = left
9. HA [LOC] = last
10. Exit

Source Code:

```

#include<stdio.h>
int arr[20],n;
//Function to display the elements in the array
void display()
{ int i;
  for(i=0;i<n;i++)
    printf ("%d ",arr[i]);
  printf ("\n");
}/*End of display()*/
//Function to insert an element to the heap
void insert(int num,int loc)
{int par;
  while(loc>0)
  {
    par=(loc-1)/2;
    if (num<=arr[par])
    {
      arr[loc]=num;
      return;
    }
    arr[loc]=arr[par];
    loc=par;
  }
}
  
```



```
        }/*End of while*/
        arr[0]=num;
    }/*End of insert()*/
    //This function to create a heap
    void create_heap()
    {
        int i;
        for(i=0;i<n;i++)
            insert(arr[i],i);
    }/*End of create_heap()*/
    //Function to delete the root node of the tree
    void del_root(int last)
    {
        int left,right,i,temp;
        i=0; /*Since every time we have to replace root with last*/
        /*Exchange last element with the root */
        temp=arr[i];
        arr[i]=arr[last];
        arr[last]=temp;
        left=2*i+1; /*left child of root*/
        right=2*i+2; /*right child of root*/
        while( right < last)
        {
            if ( arr[i]>=arr[left] && arr[i]>=arr[right] )
                return;
            if ( arr[right]<=arr[left] )
            {
                temp=arr[i];
                arr[i]=arr[left];
                arr[left]=temp;
                i=left;
            }
            else
            {
                temp=arr[i];
                arr[i]=arr[right];
                arr[right]=temp;
                i=right;
            }
            left=2*i+1;
            right=2*i+2;
        }/*End of while*/
        if ( left==last-1 && arr[i]<arr[left] )/*right==last*/
        {
            temp=arr[i];
            arr[i]=arr[left];
            arr[left]=temp;
        }
    }/*End of del_root*/
    //Function to sort an element in the heap
    void heap_sort()
    {
        int last;
        for(last=n-1; last>0; last--)
            del_root(last);
    }/*End of del_root*/
    int main()
```

```
{
    int i;
    printf ("\nEnter number of elements : ");
    scanf ("%d",&n);
    for(i=0;i<n;i++)
    {
        printf ("\nEnter element %d : ",i+1);
        scanf ("%d",&arr[i]);
    }
    printf ("\nEnter list is :\n");
    display();
    create_heap();
    printf ("\nHeap is :\n");
    display();
    heap_sort();
    printf ("\nSorted list is :\n");
    display();
}/*End of main()*/
```

7.12 Exchange Sort

The exchange sort is almost similar as the bubble sort. In fact some people refer to the exchange sort as just a different bubble sort. The exchange sort compares each element of an array and swap those elements that are not in their proper position, just like a bubble sort does. The only difference between the two sorting algorithms is the manner in which they compare the elements.

The exchange sort compares the first element with each element of the array, making a swap where is necessary. In some situations the exchange sort is slightly more efficient than its counter part the bubble sort. The bubble sort needs a final pass to determine that it is finished, thus is slightly less efficient than the exchange sort, because the exchange sort doesn't need a final pass.

Assume the following sequence of number: **45 67 78 12 9**

45	67	78	12	9
12	67	78	45	9
9	67	78	45	12
9	67	78	45	12
9	45	78	67	12
9	12	78	67	45
9	12	67	78	45
9	12	45	78	67
9	12	45	67	78

7.12.1 Algorithm

1. Compare the first pair of numbers (positions 0 and 1) and reverse them if they are not in the correct order.
 2. Repeat for the next pair (positions 1 and 2).
 3. Continue the process until all pairs have been checked.
 4. Repeat steps 1 through 3 for positions 0 through n - 1 to i (for i = 1, 2, 3, ...) until no pairs remain to be checked.
 5. The list is now sorted.
 6. Display the sorted list
 7. Exit
-

Source Code:

```
#include<stdio.h>
void display(int a[],int n);
int main(void)
{
    int array[50];           // An array of integers.
    int i, j;
    int temp;
    int n;
    printf("Enter n");
    scanf("%d",&n);
    //Some input
    for (i = 0; i < n; i++)
    {
        printf("Enter a number: ");
        scanf("%d", &array[i]);
    }
    //Algorithm
    for(i = 0; i < (n -1); i++)
    {
        for (j=(i + 1); j < n; j++)
        {
            if (array[i] > array[j])
            {
                temp = array[i];
                array[i] = array[j];
                array[j] = temp;
            }
            display(array,n);
        }
    }
    printf("\nSorted Array is: \n");
    display(array,n);
}

void display(int array[],int n)
{
    int i;
    for (i = 0; i < n; i++)
    {
        printf("%d\t",array[i]);
    }
    printf("\n");
}
```

7.13 External Sort

The Internal sorting algorithm applies to arrays that reside in the main memory. When large (voluminous) data have to be sorted they cannot bring into the main memory at the same time. In this situation, the entire file may reside on some secondary storage device. That is any sorting algorithm that uses external memory, such as tape or disk, during the sort is called **external sort**. Since most common sort algorithms assume high-speed random access to all intermediate memory, they are unsuitable if the values to be sorted do not fit in main memory. Internal sorting refers to the sorting of an array of data which is in RAM. The main concern with external sorting is to minimize external disk access since reading a disk block takes about a million times longer than accessing an item in

RAM.

The involvement of external storage device makes sorting algorithms more complex because of the following reasons:

1. The cost of accessing an item is much higher than any computational cost.
2. Different procedures and methods have to be implemented and executed for different external storage devices.

External storage devices can be categorized into two types based on the access method. They are:

- a. Sequential Access Devices (e.g., Magnetic tapes)
- b. Random Access Devices (e.g., Disks)

7.13.1 Algorithm

The sequence of steps required to perform the merge sort operation on a file are as follows:

Steps

1. The given file is divided into 2 sub-files P1 and P2.
2. Then these files P1 and P2 are compared one pair of records at a time and merged and written to the output file M1 in the sorted order.
3. Then once again, one item is read from P1 and P2 and these two records are written on sorted order in another file called M2.
4. Thus, the alternate pair of sorted items are stored in files M1 and M2.
5. This process is repeated till the sub-files P1 and P2 come to an end.
6. The following steps have to be repeated twice.
 - 6.1. Read one item from M1 and M2 and write them in sorted order in file P1.
 - 6.2. Read one item from M1 and M2 and write them in sorted order in file P2.
7. Now the following steps have to be performed 4 times.
 - 7.1. Read one item from file P1 and P2 and write the sorted records in M1.
 - 7.2. Read one item from file P1, P2 and write the sorted records in M2.
8. Stop.